

Adatbázisrendszerek II.

Féléves egyéni PL/SQL Ffladat

Készítette: Ujvári Zsombor Balázs

Neptunkód: QPQ7K4

Gyakoltatvezető neve: Dr. Bernarik László

Tartalomjegyzék

Adatbázisrendszerek II.	1
Féléves egyéni PL/SQL Ffladat	1
Készítette: Ujvári Zsombor Balázs	1
Neptunkód: QPQ7K4	1
Gyakoltatvezető neve: Dr. Bernarik László	1
1. Témakör ismertetése:	3
2. A táblákat sém szerkezetének tervezete	5
2.1 Kolcsonzo tábla létrehozása	6
2.2 Konyv tábla létrehozása	6
2.3 Kolcsonzes tábla létrehozása	7
2.4 Kolcsonzes_Naplo tábla létrehozása	7
3. A PL/SQL program kódja, megtervezése	8
4. A PL/SQL programokba kurzor, csomag, trigger -megtervezése, leírása	10
4.1 Implicit kurzor	10
4.2 Explicit kurzor	11
4.3 Tárolt csomag	12
4.4 Automatikus kulcs trigger	13
4.5 Naplózó trigger	14
4.6 Kontroll trigger	15
5. PL/SQL program extra funkcióinak rövid bemutatása, leírása	16
Felhasznált irodalom	17

1. Témakör ismertetése:

A féléves feladatom témája egy könyvtári kölcsönző rendszer. A rendszer célja, hogy nyilvántartsa a könyvtárban található könyveket, a regisztrált kölcsönzőket, valamint a könyvek kölcsönzésével kapcsolatos adatokat.

A PL/SQL feladat az első, JDBC-s feladatban elkészített adatbázisra épül. Az első feladatban a könyvtári rendszerhez tartozó táblákat és az alapvető adatkezelési műveleteket kliensoldalon, Java nyelven valósítottam meg. A második feladatban ugyanehhez az adatbázishoz készítettem szerveroldali PL/SQL programokat.

Az adatbázis három fő táblából áll. A Kolcsonzo tábla a könyvtár olvasóinak adatait tartalmazza. Ilyen adat például a kölcsönző azonosítója, neve, e-mail címe, lakcíme, tiltott státusza és a regisztráció dátuma.

A Konyv tábla a könyvek adatait tárolja. Ebben szerepel a könyv azonosítója, típusa, szerzője, címe, kiadója, kiadási ideje, a könyvtár neve és a rendelkezésre álló darabszám.

A Kolcsonzes tábla a kölcsönzéseket tartalmazza. Ez a tábla kapcsolatot teremt a kölcsönzők és a könyvek között. Egy kölcsönzéshez tartozik egy kölcsönző, egy könyv, a kölcsönzés dátuma és a leadás határideje.

A PL/SQL megoldásban tárolt eljárásokat készítettem új kölcsönzés felvitelére, meglévő kölcsönzés módosítására és kölcsönzés törlésére. Emellett tárolt függvények is készültek, amelyekkel egy adott kölcsönzés adatai lekérdezhetők, illetve megállapítható, hogy egy adott kölcsönzőnek hány kölcsönzése van.

A feladat része egy kolcsonzes_pkg nevű tárolt csomag is. Ez a csomag a Kolcsonzes táblához tartozó eljárásokat és függvényeket fogja össze. A csomag használata azért előnyös, mert a hasonló feladatokat ellátó programrészek egy helyen találhatók meg.

A megoldásban triggerek is szerepelnek. Az egyik trigger automatikusan megadja az új kölcsönzés azonosítóját. Egy másik trigger naplózza a kölcsönzésekhez kapcsolódó beszúrásokat, módosításokat és törléseket. A harmadik trigger ellenőrzi a módosításokat, például nem engedi, hogy tiltott kölcsönző új könyvet kölcsönözzön, illetve azt sem, hogy a leadási dátum korábbi legyen, mint a kölcsönzés dátuma.

A rendszer így alkalmas a könyvtári kölcsönzések egyszerű kezelésére. A PL/SQL programok segítségével az adatkezelési szabályok közvetlenül az adatbázisban valósulnak meg, így a rendszer biztonságosabban és ellenőrzöttebben működik.

2. A táblákat sémaszerkezetének tervezete

Az adatbázis egy könyvtári kölcsönző rendszer működését valósítja meg. A rendszer három fő táblából áll: *Kolcsonzo*, *Konyv* és *Kolcsonzes*. Ezek mellett létrehoztam egy *Kolcsonzes_Naplo* nevű táblát is, amely a kölcsönzésekkel kapcsolatos módosításokat tárolja.

A *Kolcsonzo* tábla a könyvtárba regisztrált olvasók adatait tartalmazza. A tábla elsődleges kulcsa a *Kolcsonzo_ID* mező. A táblában szerepel a kölcsönző neve, e-mail címe, lakcíme, tiltott státusza és regisztrációs dátuma.

A *Konyv* tábla a könyvtárban található könyvek adatait tárolja. A tábla elsődleges kulcsa a *Konyv_ID* mező. A táblában megtalálható a könyv típusa, szerzője, címe, kiadója, kiadási ideje, a könyvtár neve és a rendelkezésre álló darabszám.

A *Kolcsonzes* tábla a kölcsönzések adatait tartalmazza. A tábla elsődleges kulcsa a *Kolcsonzes_ID* mező. A tábla idegen kulcsokkal kapcsolódik a *Kolcsonzo* és a *Konyv* táblákhoz. A *Kolcsonzo_ID* mező a kölcsönzőt, a *Konyv_ID* mező pedig a kölcsönzött könyvet azonosítja.

A *Kolcsonzes_Naplo* tábla a kölcsönzésekhez kapcsolódó módosítási eseményeket tárolja. Ebbe a táblába a naplózó trigger automatikusan rögzíti, ha a *Kolcsonzes* táblában beszúrás, módosítás vagy törlés történik.

Az adatbázis kapcsolatai alapján egy kölcsönzőhöz több kölcsönzés is tartozhat, valamint egy könyv is több kölcsönzésben szerepelhet. A *Kolcsonzes* tábla ezért kapcsolótáblaként működik a kölcsönzők és a könyvek között.

```
Table KOLCSONZO created.  
  
Table KONYV created.  
  
Table KOLCSONZES created.  
  
Table KOLCSONZES_NAPLO created.
```

1.ábra: A táblák létrehozása Oracle adatbázisban.

2.1 Kolcsonzo tábla létrehozása

```
CREATE TABLE Kolcsonzo (  
  Kolcsonzo_ID NUMBER PRIMARY KEY,  
  Nev VARCHAR2(60) NOT NULL,  
  EmailCim VARCHAR2(80) NOT NULL,  
  Lakcim VARCHAR2(100) NOT NULL,  
  Tiltott NUMBER(1) DEFAULT 0 NOT NULL,  
  Regisztracio_Datuma DATE NOT NULL  
);
```

2.2 Konyv tábla létrehozása

```
CREATE TABLE Konyv (  
  Konyv_ID NUMBER PRIMARY KEY,  
  Tipus VARCHAR2(30) NOT NULL,  
  Szerzo_Neve VARCHAR2(50) NOT NULL,  
  Cim VARCHAR2(80) NOT NULL,  
  Kiado VARCHAR2(50) NOT NULL,  
  Kiadas_Ideje DATE NOT NULL,  
  Konyvtar_Neve VARCHAR2(80) NOT NULL,  
  Darabszam NUMBER NOT NULL  
);
```

2.3 Kolcsonzes tábla létrehozása

```
CREATE TABLE Kolcsonzes (  
    Kolcsonzes_ID NUMBER PRIMARY KEY,  
    Kolcsonzes_Ideje DATE NOT NULL,  
    Leadas_Ideje DATE NOT NULL,  
    Kolcsonzo_ID NUMBER NOT NULL,  
    Konyv_ID NUMBER NOT NULL,  
  
    CONSTRAINT fk_kolcsonzes_kolcsonzo  
        FOREIGN KEY (Kolcsonzo_ID)  
        REFERENCES Kolcsonzo(Kolcsonzo_ID) ,  
  
    CONSTRAINT fk_kolcsonzes_konyv  
        FOREIGN KEY (Konyv_ID)  
        REFERENCES Konyv(Konyv_ID)  
);
```

2.4 Kolcsonzes_Naplo tábla létrehozása

```
CREATE TABLE Kolcsonzes_Naplo (  
    Naplo_ID NUMBER PRIMARY KEY,  
    Tabla_Nev VARCHAR2(50),  
    Muvelet VARCHAR2(20),  
    Rekord_ID NUMBER,  
    Muvelet_Datuma DATE DEFAULT SYSDATE,  
    Leiras VARCHAR2(255)  
);
```

3. A PL/SQL program kódja, megtervezése

A PL/SQL program célja a könyvtári kölcsönzések serveroldali kezelése. A megoldás központi része a *Kolcsonzes* tábla, mivel ez kapcsolja össze a kölcsönzőket és a könyveket. A PL/SQL programban ehhez a táblához készítettem tárolt eljárásokat, tárolt függvényeket, csomagot és triggereket.

A program megtervezésekor fontos szempont volt, hogy a kötelező adatkezelési műveletek adatbázisoldalon is megvalósuljanak. Ezért készült tárolt eljárás új kölcsönzés felvitelére, meglévő kölcsönzés módosítására és kölcsönzés törlésére. Ezek az eljárások a *Kolcsonzes* táblán hajtanak végre beszúrást, módosítást és törlést.

A lekérdezési feladatokhoz tárolt függvényeket készítettem. Az egyik függvény egy adott kölcsönzés adatait kérdezi le a kölcsönzés azonosítója alapján. A másik függvény aggregált értéket számol, vagyis megadja, hogy egy adott kölcsönzőnek hány kölcsönzése van.

A programban szerepel egy *kolcsonzes_pkg* nevű csomag is. Ebbe kerültek a *Kolcsonzes* táblához tartozó tárolt eljárások és függvények. A csomag használata átláthatóbbá teszi a programot, mert az összetartozó műveletek egy helyen találhatók.

A megoldás részeként három trigger készült. Az egyik trigger automatikusan megadja az új kölcsönzés azonosítóját. A második trigger naplózza a *Kolcsonzes* táblában történő beszúrásokat, módosításokat és törléseket. A harmadik trigger ellenőrzi a kölcsönzés adatait, és hibát jelez, ha például tiltott kölcsönző próbál könyvet kölcsönözni, vagy ha a leadás dátuma korábbi, mint a kölcsönzés dátuma.

A PL/SQL programban implicit és explicit kurzor is szerepel. Implicit kurzort a módosító és törlő eljárásokban használtam a *SQL%ROWCOUNT* segítségével. Ez megmutatja, hogy az adott módosító vagy törlő utasítás hány sort érintett. Explicit kurzort a kölcsönzések listázásánál alkalmaztam, ahol a *Kolcsonzes*, *Kolcsonzo* és *Konyv* táblák összekapcsolásával jelennek meg az adatok.

A programban kivételkezelés is található. Az *EXCEPTION* blokkok feladata, hogy hiba esetén érthető üzenetet jelenítsenek meg, például ha nem létező kölcsönzést szeretnénk lekérdezni vagy törölni.

A teljes kód a mellékelt SQL fájlban található. A dokumentációban a legfontosabb programrészek kerülnek bemutatásra.

```
CREATE OR REPLACE PACKAGE kolcsonzes_pkg AS
PROCEDURE uj_kolcsonzes(...);
PROCEDURE modosit_kolcsonzes(...);
PROCEDURE torol_kolcsonzes(...);
FUNCTION kolcsonzes_adatok(...) RETURN VARCHAR2;
FUNCTION kolcsonzesek_szama(...) RETURN NUMBER;
PROCEDURE listaz_kolcsonzesek;
END kolcsonzes_pkg;
```

4. A PL/SQL programokba kurzor, csomag, trigger -megtervezése, leírása

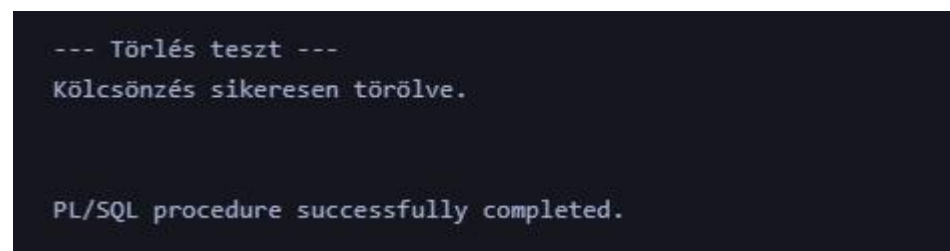
A PL/SQL megoldásban kurzor, csomag és trigger is szerepel. Ezek segítségével a program nemcsak egyszerű SQL utasításokat hajt végre, hanem összetettebb adatbázisoldali működést is biztosít.

4.1 Implicit kurzor

Implicit kurzort a módosító és törlő eljárásokban használtam. Az implicit kurzor segítségével ellenőrizhető, hogy az adott SQL utasítás hány rekordot érintett. Ehhez a `SQL%ROWCOUNT` értéket használtam.

Például törlés esetén, ha a `SQL%ROWCOUNT` értéke nulla, akkor nincs olyan kölcsönzés, amelyet törölni lehetne. Ilyenkor a program üzenetet ír ki a felhasználónak.

```
IF SQL%ROWCOUNT = 0 THEN
DBMS_OUTPUT.PUT_LINE('Nincs          ilyen          azonosítójú
kölcsönzés. ');
ELSE
DBMS_OUTPUT.PUT_LINE('Kölcsönzés sikeresen törölve. ');
END IF;
```



```
--- Törlés teszt ---
Kölcsönzés sikeresen törölve.

PL/SQL procedure successfully completed.
```

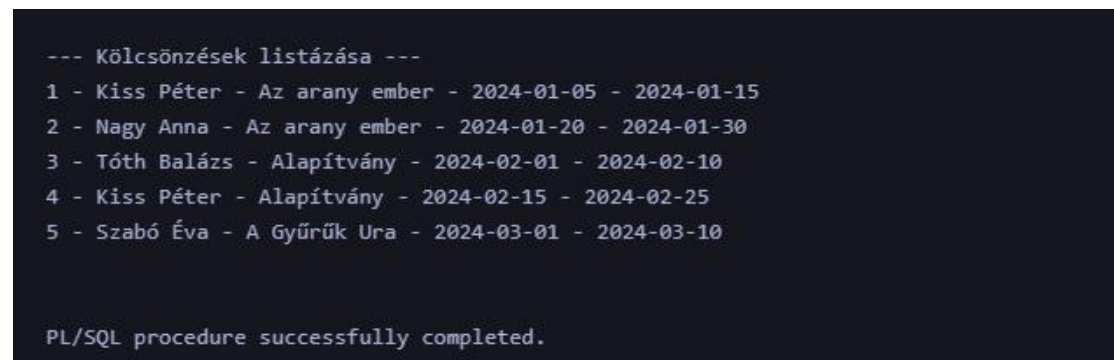
2. Ábra: Implicit kurzor használata törlés után

4.2 Explicit kurzor

Explicit kurzort a kölcsönzések listázásánál használtam. A kurzor a *Kolcsonzes*, *Kolcsonzo* és *Konyv* táblák összekapcsolásával kérdezi le a kölcsönzések adatait. Így nemcsak az azonosítók jelennek meg, hanem a kölcsönző neve és a könyv címe is.

```
CURSOR c_kolcsonzesek IS
SELECT kz.Kolcsonzes_ID,
ko.Nev,
k.Cim,
kz.Kolcsonzes_Ideje,
kz.Leadas_Ideje
FROM Kolcsonzes kz
JOIN Kolcsonzo ko ON kz.Kolcsonzo_ID = ko.Kolcsonzo_ID
JOIN Konyv k ON kz.Konyv_ID = k.Konyv_ID
ORDER BY kz.Kolcsonzes_ID;
```

Az explicit kurzor működése során a program megnyitja a kurzort, soronként beolvassa az adatokat, majd a feldolgozás végén lezárja azt.



```
--- Kölcsönzések listázása ---
1 - Kiss Péter - Az arany ember - 2024-01-05 - 2024-01-15
2 - Nagy Anna - Az arany ember - 2024-01-20 - 2024-01-30
3 - Tóth Balázs - Alapítvány - 2024-02-01 - 2024-02-10
4 - Kiss Péter - Alapítvány - 2024-02-15 - 2024-02-25
5 - Szabó Éva - A Gyűrűk Ura - 2024-03-01 - 2024-03-10

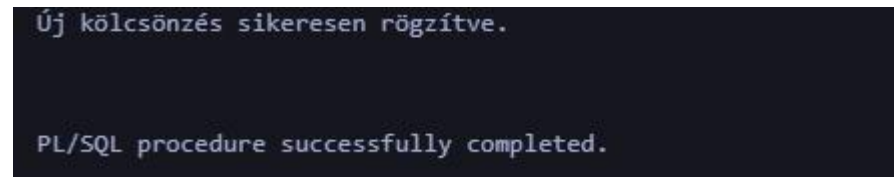
PL/SQL procedure successfully completed.
```

3. Ábra: Kölcsönzések listázása explicit kurzorral

4.3 Tárolt csomag

A *kolcsonzes_pkg* nevű csomag a *Kolcsonzes* táblához tartozó műveleteket fogja össze. A csomagban szerepelnek a beszúrást, módosítást és törlést végző eljárások, valamint a lekérdező függvények is.

A csomag előnye, hogy az összetartozó PL/SQL programrészek egy helyen találhatók. Ez átláthatóbbá és könnyebben karbantarthatóvá teszi a megoldást.



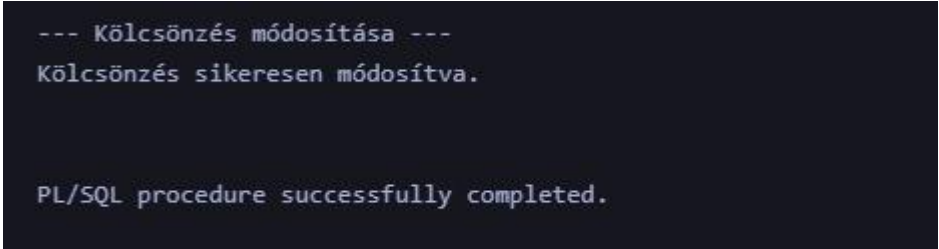
```
Új kölcsönzés sikeresen rögzítve.  
  
PL/SQL procedure successfully completed.
```

4. Ábra: A *kolcsonzes_pkg* csomag sikeres létrehozása

4.4 Automatikus kulcs trigger

Az automatikus kulcs trigger feladata, hogy új kölcsönzés beszúrásakor automatikusan megadja a *Kolcsonzes_ID* értékét, ha azt a felhasználó nem adja meg. Ehhez a trigger a *kolcsonzes_seq* sequence következő értékét használja.

```
CREATE OR REPLACE TRIGGER trg_kolcsonzes_id
BEFORE INSERT ON Kolcsonzes
FOR EACH ROW
BEGIN
IF :NEW.Kolcsonzes_ID IS NULL THEN
:NEW.Kolcsonzes_ID := kolcsonzes_seq.NEXTVAL;
END IF;
END;
/
```



```
--- Kölcsönzés módosítása ---
Kölcsönzés sikeresen módosítva.

PL/SQL procedure successfully completed.
```

5. Ábra: Automatikus kulcs trigger létrehozása

4.5 Naplózó trigger

A naplózó trigger a *Kolcsonzes* táblában történő módosításokat figyeli. Ha beszúrás, módosítás vagy törlés történik, akkor a trigger automatikusan új rekordot szúr be a *Kolcsonzes_Naplo* táblába.

A naplóban szerepel a művelet típusa, az érintett rekord azonosítója, a művelet dátuma és egy rövid leírás. Ez azért hasznos, mert így később visszanezhető, hogy milyen változások történtek a kölcsönzésekben.

```
--- Aggregált érték: kölcsönzések száma egy kölcsönzőnél ---  
A 2-es kölcsönző kölcsönzéseinek száma: 1  
  
PL/SQL procedure successfully completed.
```

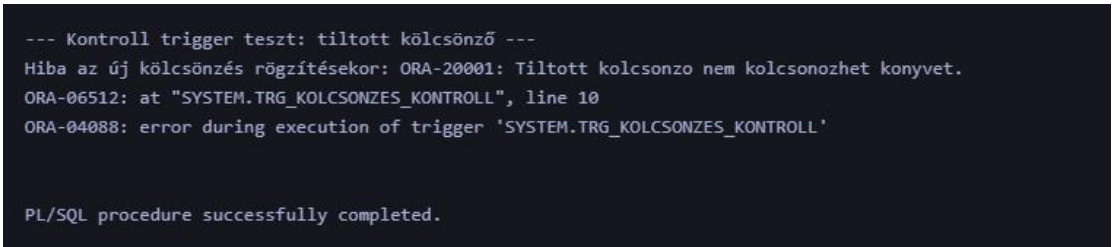
6. Ábra: Naplózó trigger működése

4.6 Kontroll trigger

A kontroll trigger feladata az adatok ellenőrzése. A trigger nem engedi, hogy tiltott kölcsönző új könyvet kölcsönözzön. Emellett ellenőrzi azt is, hogy a leadás dátuma ne legyen korábbi, mint a kölcsönzés dátuma.

Ha valamelyik feltétel nem teljesül, akkor a trigger hibát jelez a *RAISE_APPLICATION_ERROR* segítségével.

```
IF v_tiltott = 1 THEN
RAISE_APPLICATION_ERROR(
-20001,
'Tiltott kolcsonzo nem kolcsonozhet konyvet.'
);
END IF;
```



```
--- Kontroll trigger teszt: tiltott kölcsönző ---
Hiba az új kölcsönzés rögzítésekor: ORA-20001: Tiltott kolcsonzo nem kolcsonozhet konyvet.
ORA-06512: at "SYSTEM.TRG_KOLCSONZES_KONTROLL", line 10
ORA-04088: error during execution of trigger 'SYSTEM.TRG_KOLCSONZES_KONTROLL'

PL/SQL procedure successfully completed.
```

7. Ábra: Kontroll trigger hibajelzése tiltott kölcsönző esetén

5. PL/SQL program extra funkcióinak rövid bemutatása, leírása

A PL/SQL program extra funkciója a kölcsönzési műveletek automatikus naplózása. Ehhez létrehoztam egy *Kolcsonzes_Naplo* nevű táblát, amely a *Kolcsonzes* táblában történt változásokat tárolja.

A naplózás automatikusan történik egy trigger segítségével. Ha a *Kolcsonzes* táblába új rekord kerül, ha egy meglévő rekord módosul, vagy ha egy rekord törlésre kerül, akkor a trigger automatikusan beszúr egy új sort a napló táblába. A naplózott adatok között szerepel az érintett tábla neve, a művelet típusa, az érintett rekord azonosítója, a művelet dátuma és egy rövid leírás.

Ez a funkció azért hasznos, mert segítségével nyomon követhetők a kölcsönzésekkel kapcsolatos változások. A napló táblából később visszanezhető, hogy mikor történt beszúrás, módosítás vagy törlés.

A másik extra funkció az adatok ellenőrzése triggerrel. A kontroll trigger megakadályozza, hogy tiltott státuszú kölcsönző új kölcsönzést hozzon létre. Emellett azt is ellenőrzi, hogy a leadási dátum ne legyen korábbi, mint a kölcsönzés dátuma. Ez segít abban, hogy hibás vagy érvénytelen adatok ne kerüljenek az adatbázisba.

Az extra funkciók tehát biztonságosabbá és ellenőrizhetőbbé teszik a rendszert. A naplózás a változások követését segíti, az adatellenőrzés pedig megakadályozza a hibás adatok rögzítését.

Felhasznált irodalom

AI használata: 70-80%